Introduction to APIs: Wikipedia

Week 10 · APIs module · Textbook Chapter 10

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

October 19, 21 & 23, 2026

This week at a glance

We leave scraping behind and start asking servers for data *on purpose* — through a documented API.

Monday | Concepts & notebook intro

- What an API is, why it beats scraping, and why we learn on Wikipedia

Wednesday | Notebook lab

- Query the MediaWiki API: revisions, pageviews, links, raw requests

Friday | Textbook revisions |Proposal due

- Pull requests improving Chapter 10 — **and** your Final Project proposal

Companion notebook

`ch-10-wikipedia.ipynb`

Due Friday, Oct 23

Final Project proposal (5%) — a research question, your data source(s) and method, and an ethics & feasibility check.

1. Monday · Concepts

From scraping to APIs

For six weeks we **reverse-engineered** pages built for human eyes — inspecting HTML, guessing selectors, and cleaning up the mess.

This week begins Part III: **API-based data access**. Instead of taking data a site never meant to hand us, we ask for it the way the operator intends.

The skills you build here — reading documentation, constructing parameterized URLs, handling pagination, parsing nested JSON — transfer to *every* API in the rest of the course.

You have already met one: in Week 1, the pageviews API counted the readers of Wikipedia's article on CU Boulder. This week, that article is our case.

What is an API?

An **API** (Application Programming Interface) is a **contract**. It specifies what you can request. It specifies which parameters you can send. It also specifies the format of the response.

- Clean **JSON**, not cluttered HTML
- **Stable endpoints**, not fragile CSS selectors
- **Documentation** tells you exactly what exists

The tradeoff: an API only exposes what its operator *chooses* to. You cannot request a field that the API does not include. You cannot exceed the rate limits. You cannot rely on the API forever. The "post-API age" later in this module lists many APIs that operators restricted, repriced, or shut down.

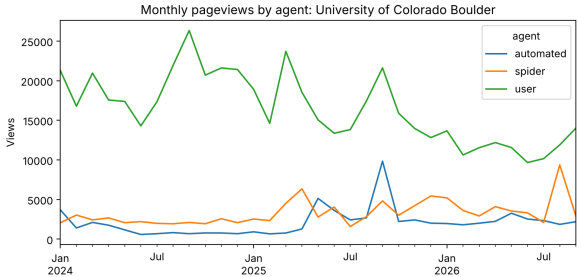
Why Wikipedia, and why now

One of the last big **open** data sources on the web: a free API, an open license, and public database dumps.

That openness is under pressure from AI:

- **Crawlers.** Bots made about 35% of pageviews but at least 65% of the costliest traffic (Wikimedia, April 2025). Rate limits by identity followed in 2026.
- **Readers.** With better bot detection, human pageviews for March–August 2025 came out about 8% lower than a year before.
- **Writing.** Editors clean up AI-written text, and a 2026 guideline bars using language models to write articles.

Our case: **University of Colorado Boulder**, beside **Boulder, Colorado**.



Monthly pageviews of the CU Boulder article, by who is reading.

A tour: what sits behind every article

Most readers see the article and nothing else. Behind it, Wikipedia keeps its records in the open:

- who changed what, and when — the **revision history**
- what editors argued about — the **talk page**
- how many people read it — **pageviews**
- what it links to, what links to it, and the same topic in other **languages**
- its **references**, **categories**, and **templates**
- the editors themselves — **user pages** and **contributions**

Each one has an **API endpoint**. Anything you can see in the interface, you can count.

The article



Tabs lead to the Article (1), its Talk page (2), and every past version (3). The languages button (4) lists other editions. A **wikilink** (5) points to another article; the **infobox** (6) comes from a **template**.

The API call

`action=query&prop=info` — the page's ID, length, and last edit

Wikilinks, and what links here

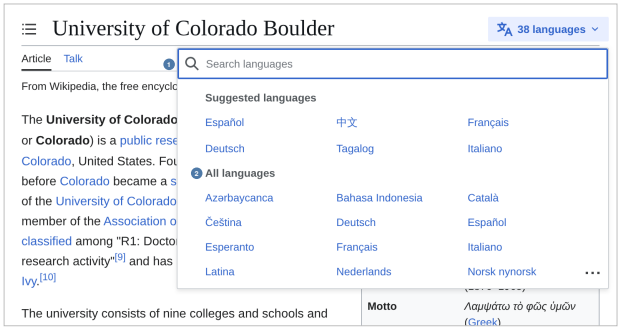


A wikilink is a citation of another article. **What links here** is the reverse map: every page that links to this one (1), with filters (2).

The API call

Links out: `prop=links`
Links in: `list=backlinks`

Interlanguage links



The same topic in other editions (2), searchable (1). Each edition is written by its own community — they are not translations.

The API call

```
prop=langlinks
```

How many editions carry a topic is a rough measure of global attention — one that skews toward the Global North.

References



1 References [edit]

1. ^ "Let Your Light Shine". University of Colorado Boulder Arts and Sciences Magazine. July 5, 2017. Archived from the original on October 8, 2017. Retrieved October 7, 2017.
2. ^ "U.S. and Canadian 2025 NCSE Participating Institutions Listed by Fiscal Year 2025 Endowment Market Value" (XLSX). National Association of College and University Business Officers (NACUBO). Retrieved February 19, 2026.
3. ^ *University of Colorado Faculty and Staff, Fall 2025*, University of Colorado System, retrieved October 14, 2026
4. ^ a b c "Fast Facts". *For Media*. University of Colorado Boulder. Retrieved October 14, 2026.
5. ^ "More Buffs are crossing the finish line—and they're getting there faster". *CU Boulder Today*. University of Colorado Boulder. September 10, 2026.
6. ^ "University of Colorado Boulder". Archived from the original on November 7, 2021. Retrieved November 7, 2021.
7. ^ *University of Colorado at Boulder NIL Brand Guidelines* (PDF). January 28, 2022. Retrieved July 28, 2023.
8. ^ "Campus, College & School Names". University of Colorado Boulder. May 11, 2017. Archived from the original on February 15, 2020. Retrieved December 4, 2019.
9. ^ "Carnegie Classifications Institution Lookup". *carnegieclassifications.iu.edu*. Center for Postsecondary Education. Archived from the original on July 24, 2020. Retrieved July 24, 2020.
10. ^ Greene, Howard R.; Greene, Matthew W. (2001). *The Public Ivies: America's Flagship Public Universities*

An article's claims rest on its references (1): numbered citations (2), many with an archived copy (Week 3).

The API call

`prop=extlinks` — every external link on the page

Categories and templates



Categories (1) file the article with others like it. Like the infobox's template, they are kept by editors — as consistent as the people who keep them.

The API call

```
prop=categories
prop=templates
```

The talk page

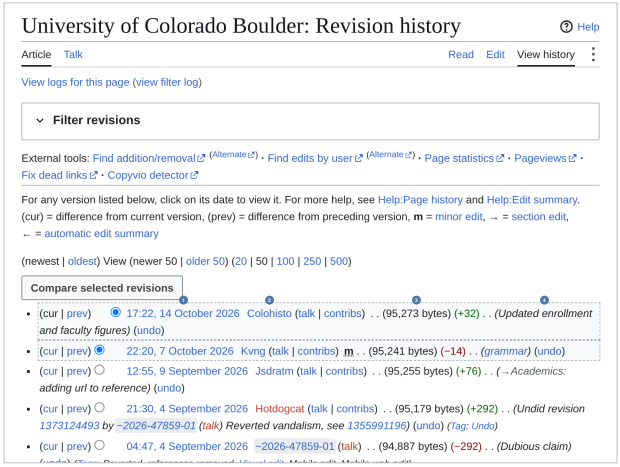


Where editors discuss changes (1). Its banners (2) record the article's quality rating and the WikiProjects that look after it.

The API call

Any of the same calls, for `titles=Talk:...`

The revision history



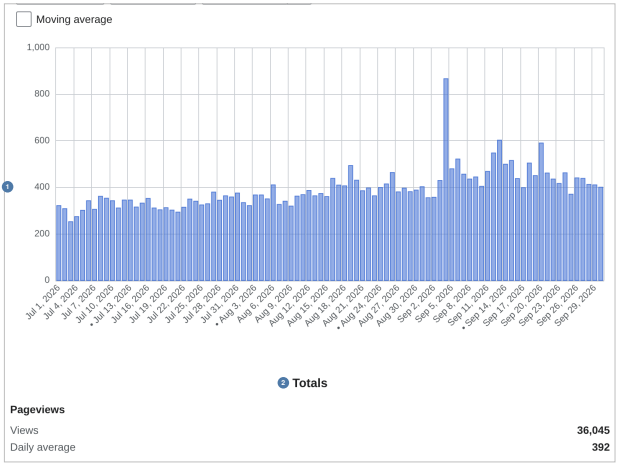
Every edit ever made, one row each: when (1), by whom (2), how many bytes (3), and the editor's summary (4).

The API call

`prop=revisions` — with `rvstart` to fix the window

Usernames are public; many are pseudonyms.

Pageviews



Wikipedia counts its readers and publishes the counts: daily views (1) and the period's total (2), here for July–September 2026.

The API call

A second API (REST):
`/metrics/pageviews/per-article/...`

A pageview is not a person, and the definition has changed. Wednesday shows how.

User pages and contributions



User contributions for Jimbo Wales

Help

Results for Jimbo Wales (talk | block log | uploads | logs | global block log | global account | filter log)
A user with 15,133 edits. Account created on 27 March 2001.

Search for contributions

(newest | oldest) View (newer 50 | older 50) (20 | 50 | 100 | 250 | 500)

- 12:19, 30 September 2026 (diff | hist) . . (+377) . . User talk:Jimbo Wales (→New cross-language page link: Shotgun: Reply) (Tag: Reply)
- 19:50, 28 September 2026 (diff | hist) . . (+143) . . User talk:Jimbo Wales (→New cross-language page link: Shotgun: Reply) (Tag: Reply)
- 09:43, 27 September 2026 (diff | hist) . . (+496) . . User talk:Jimbo Wales (→Question about Deletionism and inclusionism: Reply) (Tag: Reply)
- 16:35, 25 September 2026 (diff | hist) . . (+218) . . User talk:Jimbo Wales (→Off Topic Question: Reply) (Tag: Reply)
- 16:35, 25 September 2026 (diff | hist) . . **(+550)** . . User talk:Jimbo Wales (→Question about using AI: Reply) (Tag: Reply)
- 18:50, 19 September 2026 (diff | hist) . . **(+603)** . . User talk:Jimbo Wales (→Update on Staff Union Vote and information from WMF: Reply) (Tag: Reply)
- 18:47, 19 September 2026 (diff | hist) . . (+385) . . User talk:Jimbo Wales (→Update on Staff Union Vote and information from WMF: Reply) (Tag: Reply)
- 21:56, 18 September 2026 (diff | hist) . . **(+1,034)** . . User talk:Jimbo Wales (→Update on Staff Union Vote and information from WMF: Reply) (Tag: Reply)
- 21:51, 18 September 2026 (diff | hist) . . **(+2,210)** . . User talk:Jimbo Wales (→Update on Staff Union Vote and

Editors have pages too. Jimbo Wales's contributions, across all of Wikipedia: when (1), which page (2), how many bytes (3), and the summary (4).

The API call

`list=usercontribs`
(the user page itself: `titles=User:Jimbo_Wales`)

From the interface to the API

What you see	Where	API request
The article's basic facts	the article	`prop=info`
Wikilinks out	the article's text	`prop=links`
What links here	Tools → What links here	`list=backlinks`
Templates	the infobox and other boxes	`prop=templates`
Categories	the foot of the article	`prop=categories`
References' links	References	`prop=extlinks`
Other languages	the languages button	`prop=langlinks`
Talk page	the Talk tab	any of these, for `titles=Talk:...`
Revision history	View history	`prop=revisions`
Pageviews	Page information	REST `/metrics/pageviews/...`
User contributions	Special:Contributions	`list=usercontribs`

Every Action API request here starts
`https://en.wikipedia.org/w/api.php?action=query&format=json`

The anatomy of a request — and what the notebook covers

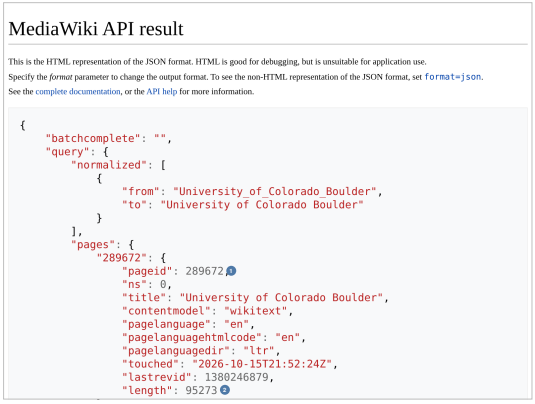
Every MediaWiki call is four parts:

1. a **base URL** — `.../w/api.php`
2. an `action` — what you want to do
3. **parameters** that narrow the request
4. a `format` — `json`

The response is nested JSON you navigate **one level at a time**:
`data["query"]["pages"][page_id]`.

Connecting to Ch. 2 (Ethics)

Identify yourself: a **User-Agent** with your contact information. Since 2026, Wikimedia gives an identified client 200 requests a minute, and an anonymous one 10.



The same request in a browser, with `format=jsonfm`: the page ID (1) is the key under `pages`, and `length` (2) is the article's size in bytes.

Your Final Project proposal, due Friday

Three parts, worth 5% of your grade:

1. **A research question.** What the data could answer, and who would care about the answer. A question, not a topic: not "Wikipedia," but "Which events send readers to articles about wildfires?"
2. **Data source(s) and method.** The site, API, or documents, and how you'll get them: an API, scraping, PDFs, or a mix. Name the endpoint, page, or file.
3. **Ethics and feasibility.** The terms of service, `robots.txt`, and any personal data in it — and proof that you can get **one record**: paste the request and what came back.

This week makes step 3 easy

If your source has an API, Wednesday's notebook is your feasibility check: one request, one record, the status code, and the fields you'll use.

Stuck for an idea?

Week 9's Friday slides grow every chapter's *Additional Exercises* into project questions.

Collecting over time?

Start the collector now, and say so in the proposal.

2. Wednesday · Notebook Lab

The four parts, then navigate the JSON

Assemble the request as a dictionary of parameters; `requests` builds the URL for you:

```
import requests
url = "https://en.wikipedia.org/w/api.php"
HEADERS = {"User-Agent": "WebDataScience/1.0 (your-email@colorado.edu)"}

params = {"action": "query", # what to do
          "titles": "University of Colorado Boulder", # about this page
          "prop": "info", # what we want back
          "format": "json"} # how we want it
data = requests.get(url, params=params, headers=HEADERS).json()
```

Then walk the nested response one key at a time — page IDs are the keys:

```
pages = data["query"]["pages"]
page_id = list(pages.keys())[0]
print(pages[page_id]["title"], pages[page_id]["pageid"], pages[page_id]["length"])
```

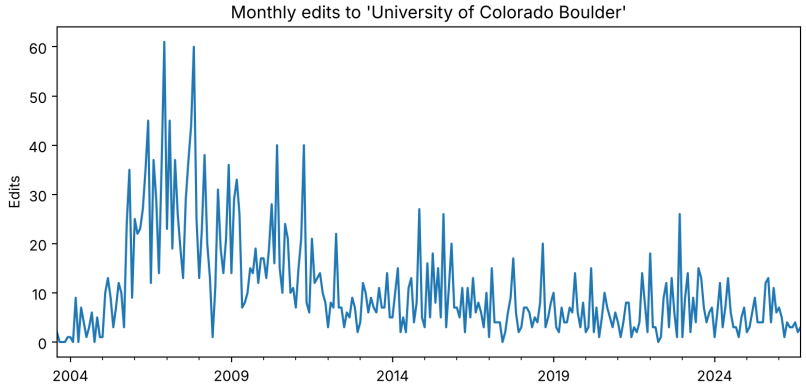
Revisions — and the pagination loop

```python
def get_revisions(title, until="2026-09-30T23:59:59Z"):
    params = {"action": "query", "titles": title,
        "prop": "revisions", "rvlimit": "max",
        "rvprop": "ids|timestamp|user|size|comment",
        "rvstart": until, # newest edit to include
        "format": "json"}
    all_revisions = []
    while True: # the pagination loop
        response = requests.get(url, params=params,
                                headers=HEADERS)
        response.raise_for_status()
        data = response.json()
        page = list(data["query"]["pages"].values())[0]
        all_revisions.extend(page.get("revisions", []))
        if "continue" in data:
            params.update(data["continue"]) # next batch
        else:
            break
    return pd.DataFrame(all_revisions)
```

The most common API bug: **forgetting to paginate**. The server returns 500 at a time and a `continue` token when more remain. Skip the loop and you silently keep a *fraction*.

`rvstart` fixes the window, so next week's run gives today's answer.

Every edit since 2003



2,988 revisions through September 2026; the ten most active editors made 15.9% of them. The busiest years came early, when the article was being written; since then, a few edits a month keep it current.

Pageviews: a second API

The REST API puts its parameters in the URL's *path*. Define the function once:

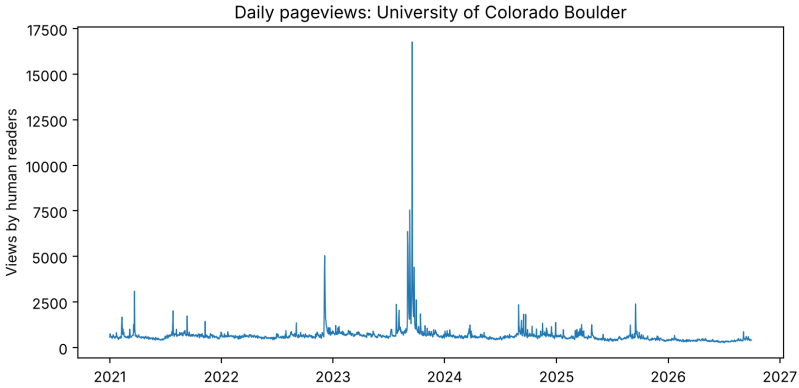
```
def get_pageviews(title, start, end,
                  agent="user", granularity="daily"):
    article = title.replace(" ", "_")
    url = ("https://wikimedia.org/api/rest_v1/metrics"
           "/pageviews/per-article/en.wikipedia"
           f"/all-access/{agent}/{article}"
           f"/{granularity}/{start}/{end}")
    response = requests.get(url, headers=HEADERS)
    response.raise_for_status()
    views = pd.DataFrame(response.json()["items"])
    views["timestamp"] = pd.to_datetime(
        views["timestamp"], format="%Y%m%d%H")
    return views[["timestamp", "views"]]
```

Define the function in one cell, and use it in the next:

```
cu_views = get_pageviews(
    "University of Colorado Boulder",
    "20210101", "20260930")
```

No `continue` here: one request returns the whole range.

Spikes are events

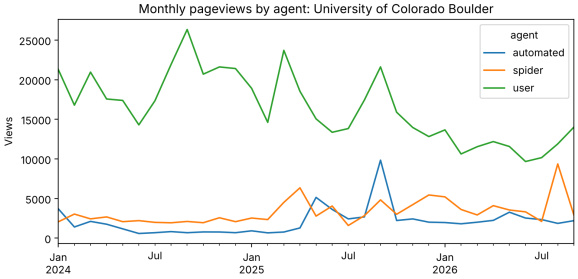


A typical day: about 600 human readers. December 5, 2022, two days after Deion Sanders was hired: 5,042. September 17, 2023, after a double-overtime win over Colorado State that ended late on the 16th: 16,767 — the API's days are **UTC** days.

What a pageview counts

A pageview is **not a person**: one reader who reloads three times is three views.

- The REST API's counts start in **July 2015**, under a definition drafted in 2014–15 (Research:Page view).
- Before that, pagecounts-raw (2007–2016) counted every request, edits and crawlers included. A fallback, not comparable.
- The **agent** says who is reading: `user` (people), `spider` (crawlers that say so), `automated` (bots that don't).
- When bot detection improves, past counts can be **reclassified** — as March–August 2025 were.



The same function, called once for each agent.

One function for every other feature

Links, categories, templates, languages, backlinks, and contributions are all `action=query` with a different `prop` or `list`. Write the continuation loop once:

```
def query_all(params):
    params = {"action": "query", "format": "json", **params}
    results = []
    while True:
        response = requests.get(url, params=params, headers=HEADERS)
        response.raise_for_status()
        data = response.json()
        results.append(data["query"])
        if "continue" in data:
            params.update(data["continue"])
        else:
            break
    return results

# page_list() runs query_all() for one page and one prop, and returns the items
categories = page_list(title, "categories", "cllimit")
backlinks = query_all({"list": "backlinks", "bltitle": title, "bllimit": "max"})
```

From raw requests to a client: mwclient

```python
import mwclient

site = mwclient.Site("en.wikipedia.org",
    clients_useragent="WebDataScience/1.0 (your-email@colorado.edu)")
page = site.pages["University of Colorado Boulder"]

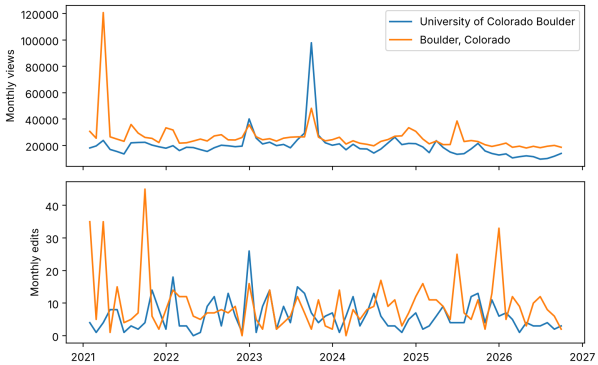
print(len(list(page.categories())))
print(len(list(page.langlinks())))
revisions = list(page.revisions(
    start="2026-09-30T23:59:59Z", api_chunk_size=500))
```

Each line replaces one of our functions — continuation included — and mwclient backs off when the servers report lag. A **429** still stops it (version 0.11): wait, and run it again.

Two things stay raw: **pageviews** (mwclient speaks the Action API only), and any parameter the library doesn't expose.

mwclient docs · `pip install mwclient`

Combining endpoints: the university and the city



The same functions, on **Boulder, Colorado**: monthly views (top) and edits (bottom), side by side.

Do articles get edited more when more people read them? Which events move the university's article, and which move the city's?

Lab: work these, then show-and-tell

Work in pairs. Do these exercises from the chapter:

1. Compare the revision histories of two related articles. Report the unique editors, the edits for each editor, and the monthly activity.
2. Get the daily pageviews of five articles. Report if the spikes occur together or separately.
3. Find the outgoing wikilinks of three related articles. Report the links that they share.
4. Read the API documentation. Write a request for a new endpoint such as `extlinks` or `recentchanges`.
5. Find the 20 most frequent editors of an article. Then find the other articles that they edit.
6. **5617**: reproduce one measure from Mesgari et al. (2015). For example, calculate a Gini coefficient of the edits for each editor

Show-and-Tell

Bring one of these to class:

- a bug that you could not solve
- data that you collected and find interesting
- a question for a research design

Error flavors

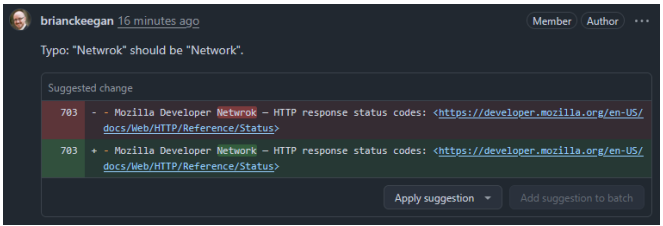
A **429** is Wikimedia's rate limit, which a whole classroom on one network can reach together: wait the `retry-after` seconds and run the cell again. No GitHub issue. A cell that fails the same way after waiting may be *genuinely* broken: confirm it, then search and file an issue.

Scraping asks: *how is this page built?*
An API asks: *what does the documentation promise?*

Learn one API by hand,
and you have learned them all.

3. Friday · Textbook Revisions

Improving Chapter 10 together



A review suggestion on the instructor's demonstration pull request: the author can apply it with one click.

Today we make the book better. Each of you opens a **pull request** against `ch-10-wikipedia.qmd` and reviews a classmate's.

The workflow (from Week 1):

1. Branch / fork the book repo
2. Edit the chapter; commit with a clear message
3. Open a PR describing *what* and *why*
4. Review two peers' PRs: kind, specific, actionable

Also due today — Fri, Oct 23

Your **Final Project proposal** (5%): a research question, your data source(s) and method, and an ethics & feasibility check — with one record you actually retrieved.

Before you submit the proposal

- ☐ Is the question one the data can answer — and would anyone outside this class care?
- ☐ Does it name the exact source: an endpoint, a site's pages, or a set of PDFs?
- ☐ Did you retrieve **one real record**, and paste the request and the response?
- ☐ Did you read the terms of service and `robots.txt`? Does the data include people's names or other personal information, and how will you handle it?
- ☐ If the data changes over time, is your collector already running?
- ☐ Working in a pair? Ask the instructor before you submit.

What comes next

Presentation, about 5 minutes (10%), in the final week, Nov. 30–Dec. 4. Repository and a 2,000–3,000-word write-up (25%), due Monday, December 7.

Late work

Loses 1% of its maximum value for each day after the deadline, unless you have an approved excuse.

A revision menu for Chapter 10

High-value targets — pick one and scope it to a single PR:

- **Close a gap in "Common Issues to Debug."** Add a worked **missing-page** check (a bad title returns page ID `-1`), or a `urllib.parse.quote()` example for titles with parentheses / accents.
- **Show the etiquette, don't just name it.** The chapter mentions `maxlag` but never demonstrates it — add a small retry-on-`maxlag` example for bulk jobs.
- **Add a figure.** A diagram of the request anatomy, or of the request → `continue` → request pagination loop, would anchor the two hardest ideas.
- **Strengthen an exercise.** Exercises 1–5 have no expected output or starter cell — add a rubric or a self-checking assertion.
- **Deepen "Social History."** Turn the gender-gap discussion into a reproducible measurement (tie it to WikiProject Women in Red) using this chapter's helper functions.

A good revision, and a good review

A strong PR

- One focused change, clearly titled
- Explains the problem it fixes
- Builds (`quarto render`) without errors
- Matches the book's voice and notation
- Discloses any AI assistance

A strong review

- Runs / reads the change, not just skims
- Names specifics (line, claim, code)
- Separates "must fix" from "nice to have"
- Is generous — assume good faith
- Ends with a clear verdict

Next week: **Government APIs —** the Census, the Federal Reserve (FRED), and other public data services.

Key takeaways

1. An API is a **documented contract**: structured, intentional access — clean JSON and stable endpoints, no reverse-engineering.
2. The skill transfers: **read the docs** → **build a parameterized request** → **parse nested JSON** one level at a time.
3. Always **paginate**: follow the `continue` token in a loop, or you silently keep only the first batch.
4. Be a good client: a descriptive **User-Agent**, serial requests, and `maxlag` for bulk jobs (the ethics of Ch. 2, applied). A client such as **mwclient** does the repetitive parts once you know what they are.
5. Know what your numbers count: a **pageview** is not a person, and its definition — and the bots behind it — have changed.
6. Openness is a *choice*, not a guarantee of completeness — Wikipedia's data still inherits its volunteer community's biases.